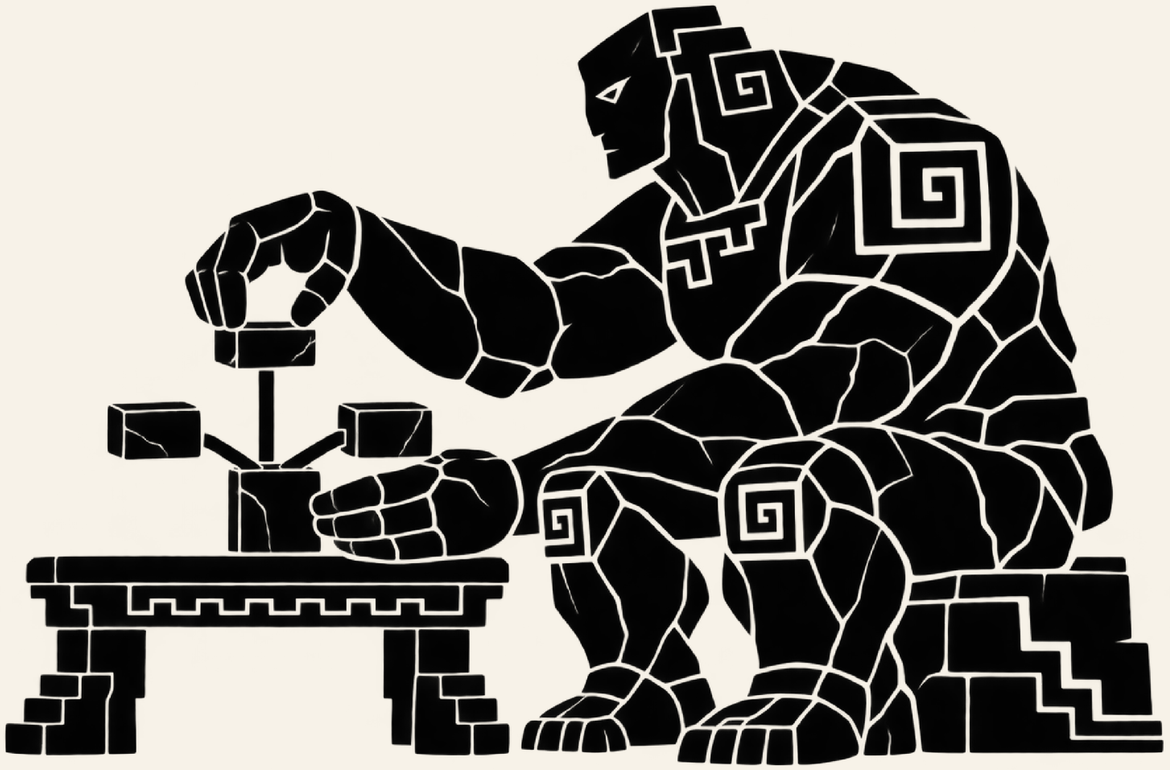
FEDOT.Industrial & FEDOT.LLM

From signal representations
to executable experiments

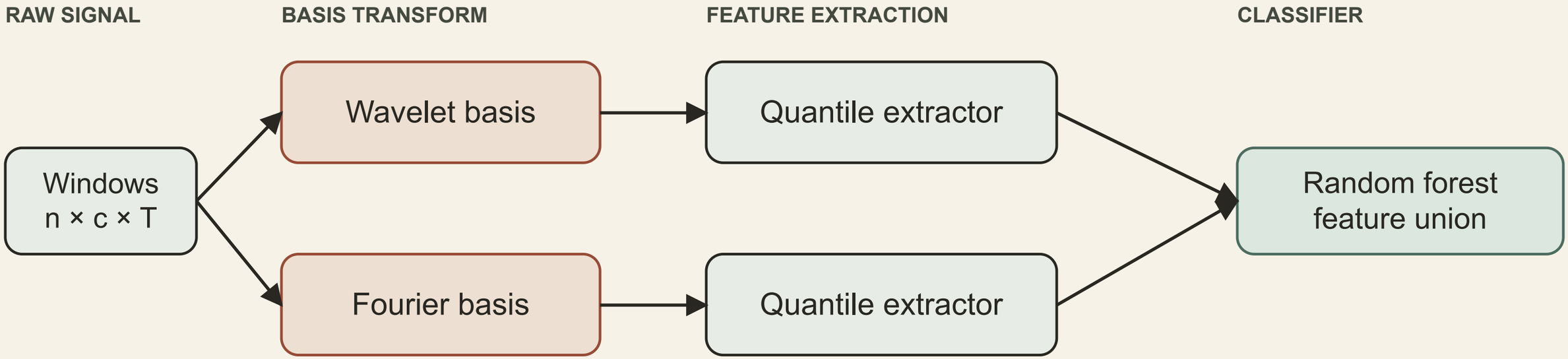


What is searched —
and what is orchestrated

Lopatenko George
AI Researcher @NSSLab

Representations become graph operations

$$X \in \mathbb{R}^{n \times c \times T} \xrightarrow{\phi_m} Z_m \in \mathbb{R}^{n \times d_m} \xrightarrow{h} \hat{\mathbf{y}}$$

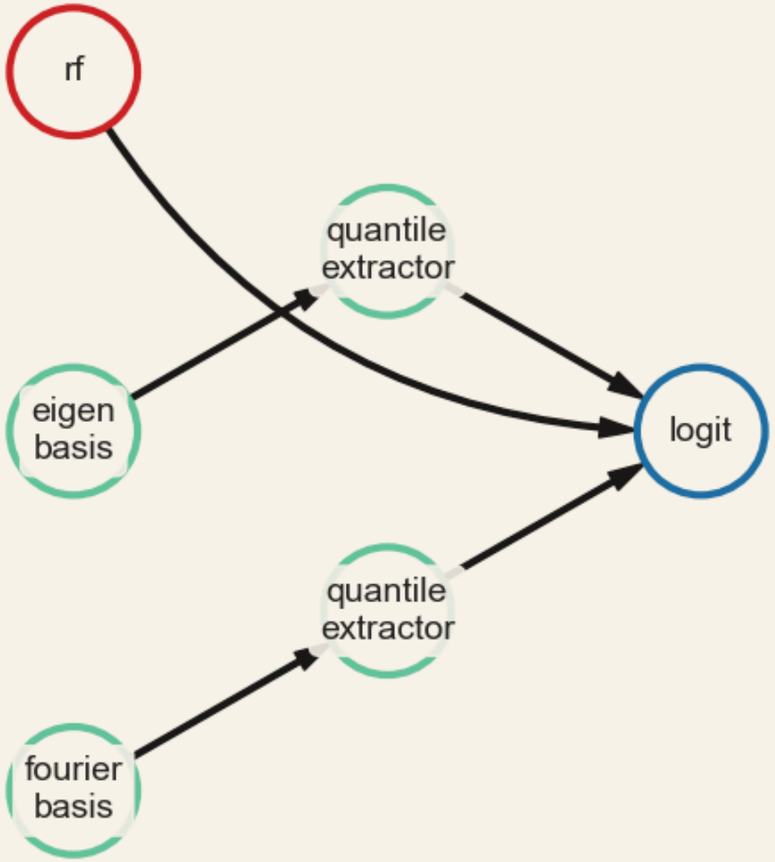


AutoML can test which signal properties make the downstream learning problem easier.

Search variables: basis family · window parameters · feature operator · model · branch structure

A graph can merge features and model outputs

Lightning2 · published UCR classification example · 6 operations / 3 input branches



MODEL RESPONSE

Random forest sends its learned output to the final estimator.

TRANSFORMED FEATURES

Eigen → quantiles
Fourier → quantiles

MIXED INPUT AT THE ROOT

The root combines feature vectors and an upstream model output.

The root merges transformed features with an upstream model response.

Kernel learning selects representations

RBF illustration

$$K_m[i,j] = \exp\left(-\gamma_m \|\phi_m(x_i) - \phi_m(x_j)\|_2^2\right), \qquad K_w = \sum_{m=1}^{M} w_m K_m$$

Learn the mixture weights

$$\max_{\mathbf{w}\in\Delta_M} \; \mathbf{a}^\mathsf{T}\mathbf{w} - \lambda\,\mathbf{c}^\mathsf{T}\mathbf{w} - \mu\,\mathbf{w}^\mathsf{T} R \mathbf{w}, \qquad \Delta_M = \{\mathbf{w} \geq 0 : 1^\mathsf{T}\mathbf{w} = 1\}$$

a / label alignment

Centred similarity to the target kernel

c / kernel complexity

Trace divided by Frobenius norm

R / redundancy

Pairwise alignment between kernels

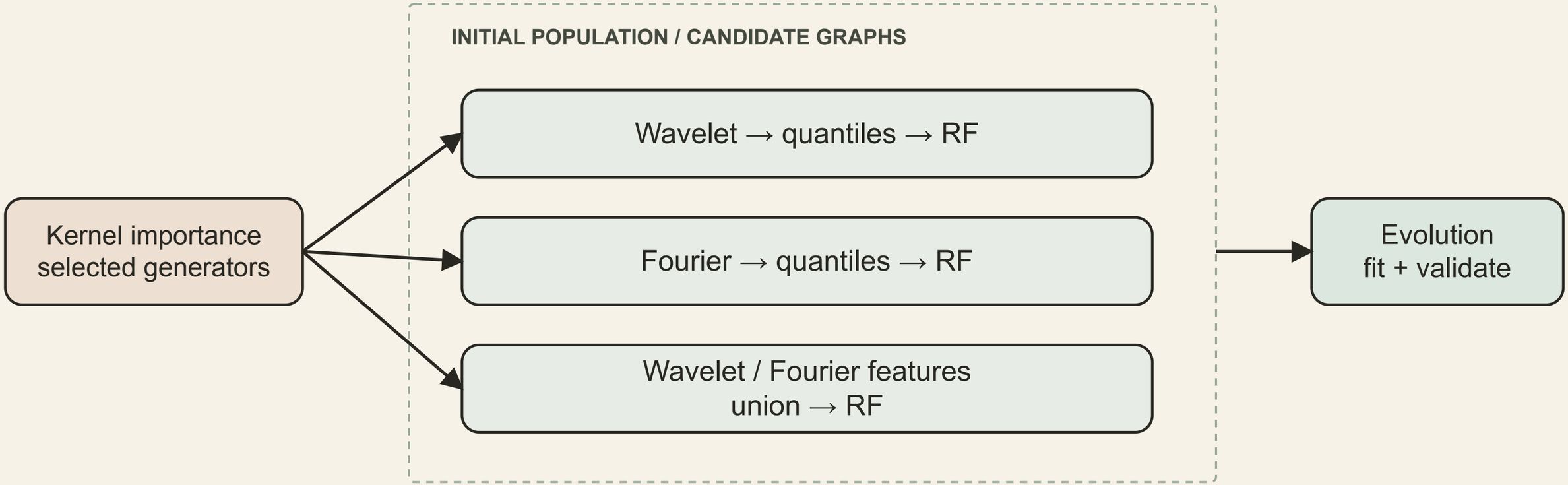
Each update: project onto the simplex → threshold → renormalise

Transfer useful, less redundant representations into the initial graph population.



Kernel weights initialise graph search

Generator weights determine seeds; the composer still evaluates complete graphs.

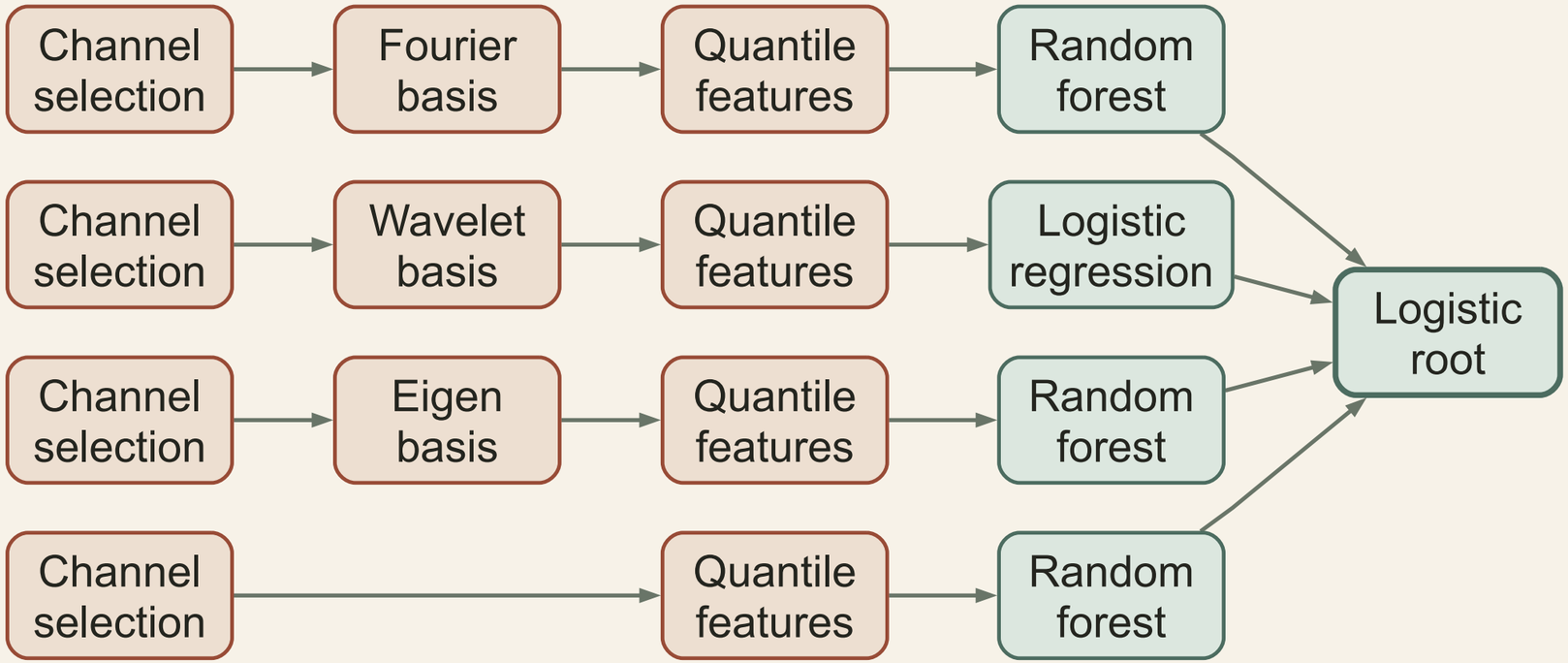


Start with informed hypotheses; spend the remaining budget evolving and validating them.

Generators without a supported mapping are excluded from this initial population.

A fitted graph with 16 operations

Complementary branch predictions become features for the final learner.



$$\boldsymbol{P} = \left[\boldsymbol{P}^{(1)} \mid \boldsymbol{P}^{(2)} \mid \boldsymbol{P}^{(3)} \mid \boldsymbol{P}^{(4)}\right] \in \mathbb{R}^{N\times 12} \xrightarrow{\text{logistic root}} \widehat{\boldsymbol{P}} \in \mathbb{R}^{N\times 3}$$

Manual graph · 90 train / 30 test · 2 channels × 128 samples · saved model reloaded

FEDOT.LLM: from request to model

01 / REQUEST

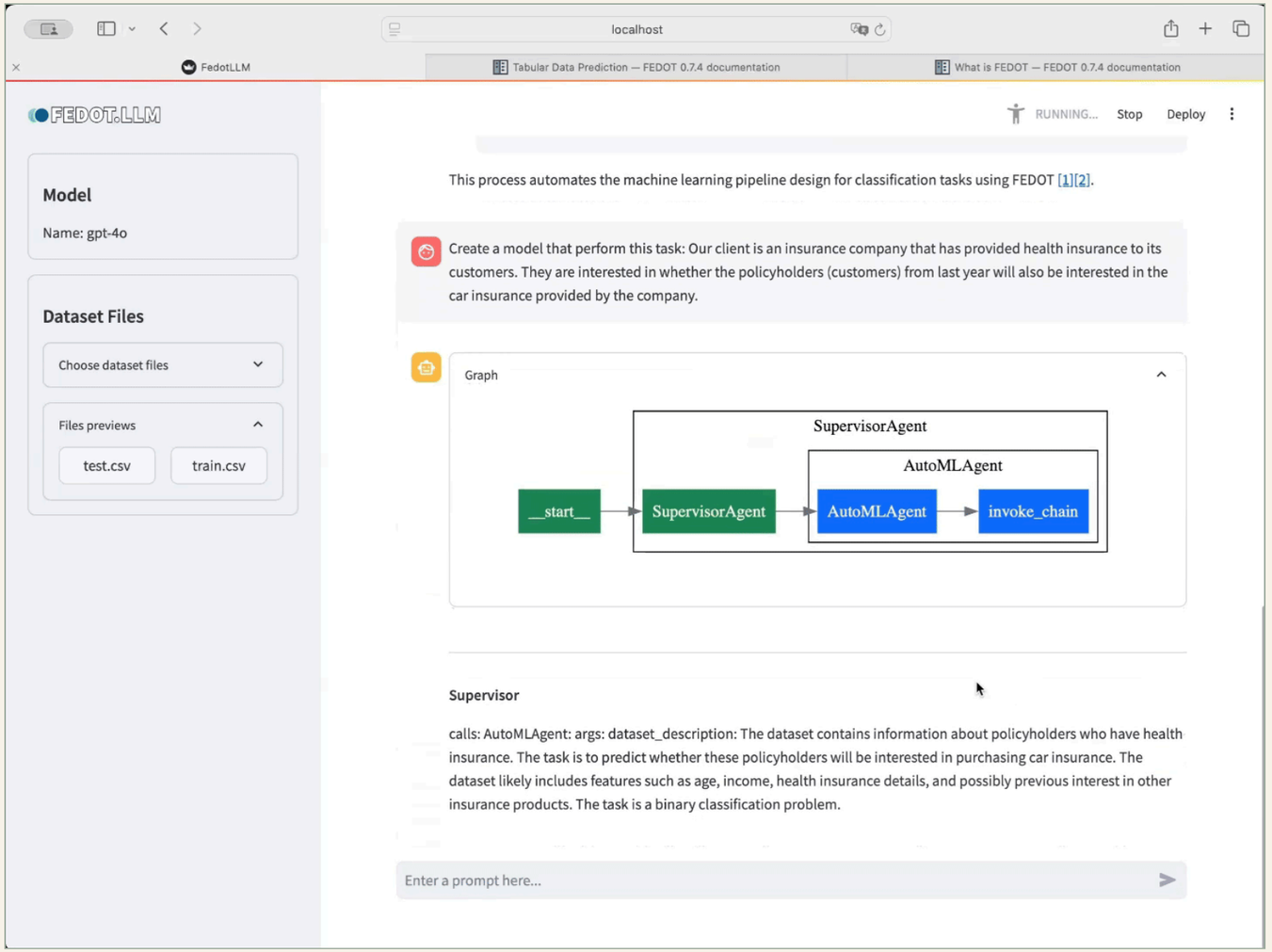
Describe the prediction task and provide the data files.

02 / AUTOML EXECUTION

The agent prepares the run; FEDOT searches models.

03 / RESULT

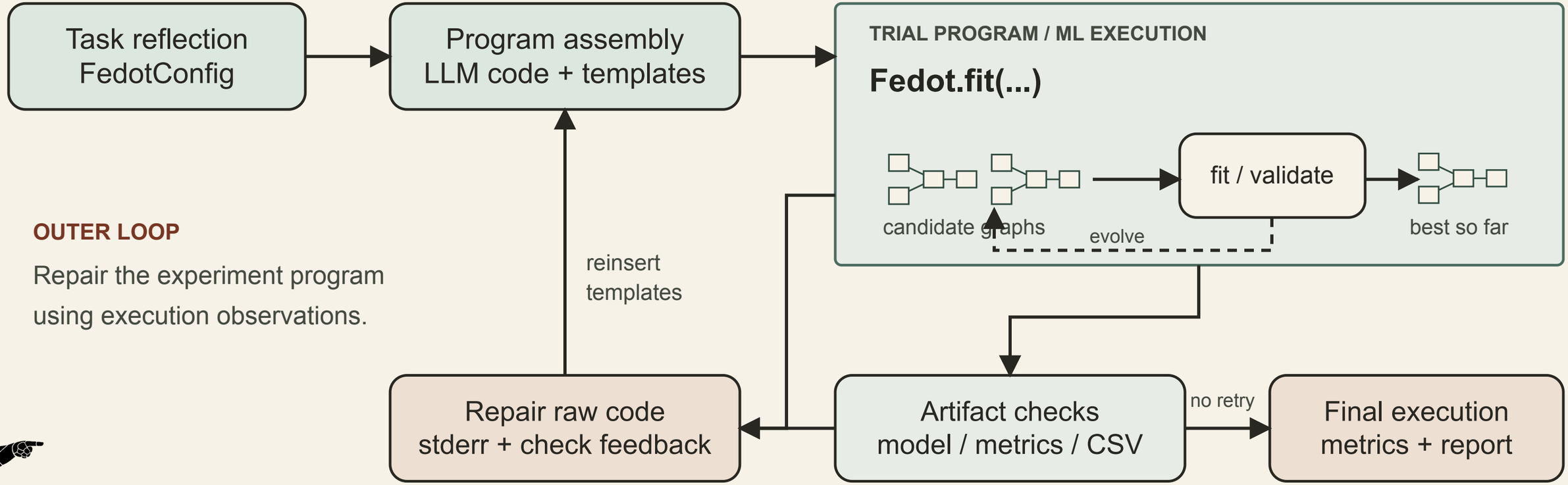
Inspect the pipeline and reported metrics.



Language prepares the task; model training and validation remain executable ML steps.

Code repair wraps the FEDOT search

Two optimisation objects: program correctness outside, model quality inside.



Automatic repair can turn execution failures into usable AutoML runs.

Default: 5 repair attempts. Retry exhaustion can still leave an error; a final run follows separately.

The search must optimise the right objective

Insurance task: rank policyholders by their probability of purchasing vehicle insurance.

SCHEMA-CONSTRAINED

```
{
  "problem": "classification",
  "timeout": 5.0,
  "cv_folds": 3,
  "preset": "best_quality",
  "metric": "roc_auc",
  "predict_method": "predict_proba"
}
```

Solver setup

TASK-SPECIFIC PYTHON

```
def transform_data(df):
    y = (df["Response"].to_numpy()
         if "Response" in df else None)
    X = df.drop(columns=["id", "Response"],
                errors="ignore")
    return X.to_numpy(), y

ids = test["id"].to_numpy()
```

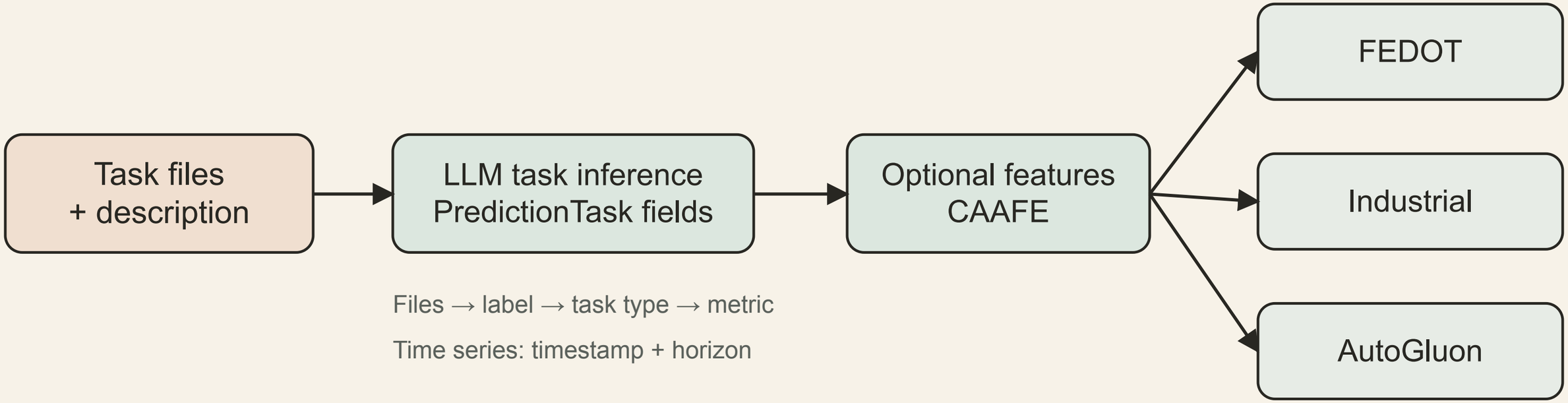
Target extraction + feature matrix + IDs

Ranking requires probability scores + ROC AUC; the code must isolate the correct target.



Fedot-assistant can add semantic features

Optional CAAFE proposes features from column meaning; measured quality decides their value.

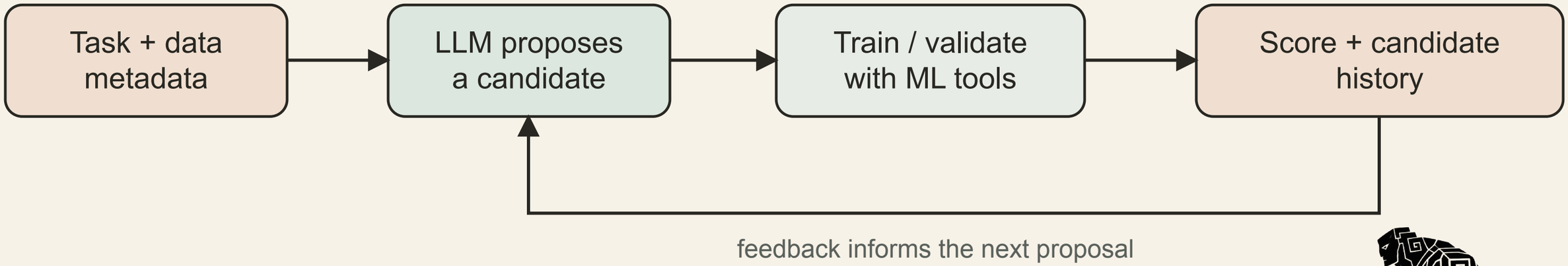


Backend selected by configuration: `automl.enabled: fedot | fedot_ind | autogluon`

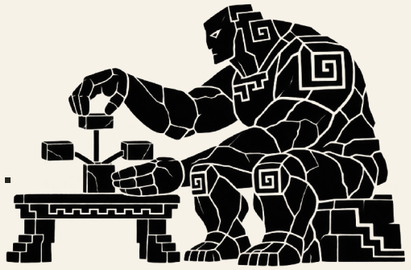
Multiple adapters are available; the orchestrator runs one configured backend.

Can LLM proposals save candidate evaluations?

Proposals depend on task context, data analysis and the history of measured candidate quality.



Candidate: configuration, feature strategy or pipeline. Evaluation still requires an ML run.



Proposed search mechanism; the implemented AutoMLAgent instead generates and repairs experiment code.

What counts as an AutoML improvement?

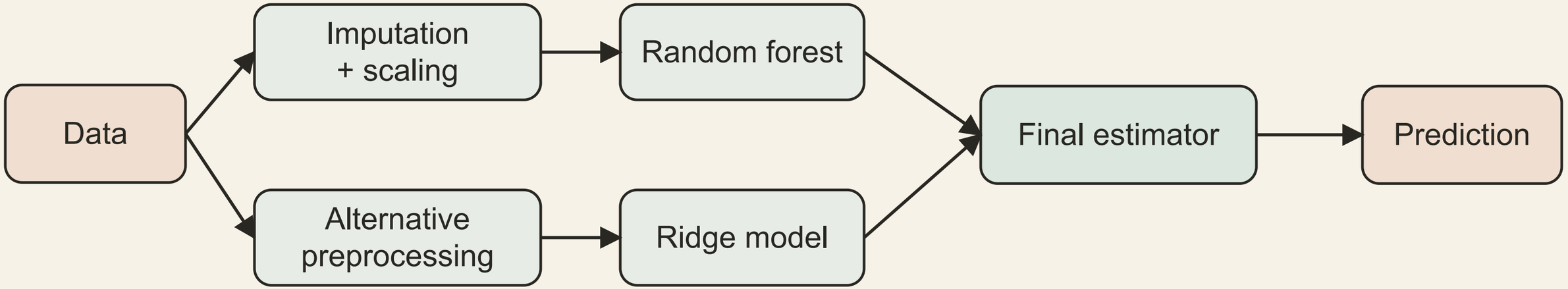
Use FEDOT as the controlled baseline; isolate the contribution of each extension.

Hypothesis	Hold fixed	Measure
Industrial Representation effect	Data split Composer Training budget	Held-out quality Runtime Selected operators
FEDOT.LLM Orchestration effect	Task AutoML backend Training budget	Successful-run rate Quality + total time LLM cost + manual fixes

Industrial changes candidate models. FEDOT.LLM changes how experiments are specified and executed.

Backup: FEDOT: search over pipeline graphs

A candidate is a directed acyclic graph of operations, with parameters at its nodes.



$$(G^*, \theta^*) \approx \underset{G \in \mathcal{G},\, \theta \in \Theta_G}{\arg\min}\ \widehat{\mathcal{L}}_{\text{val}}(f_{G,\theta}), \qquad t_{\text{search}} \leq B$$

subject to a search budget and graph constraints

Graph evolution → fit and validate candidates → select → tune hyperparameters



Backup: signal features and their roles

Several feature families preserve different information from the same window.

Global amplitude

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{T}\sum_{t=0}^{T-1} x_t^2}$$

Frequency components

$$X_k = \sum_{t=0}^{T-1} x_t \exp\left(-\frac{2\pi \mathrm{i} k t}{T}\right), \qquad f_k = \frac{k f_s}{T}$$

Local structure at scale a

$$W_x(a,b) = \frac{1}{\sqrt{|a|}} \int x(t)\, \psi^*\left(\frac{t-b}{a}\right)\, \mathrm{d}t$$

Distance to a short motif

$$\phi_s(x) = \min_u \sqrt{\frac{1}{c\ell}\sum_{q=1}^{c}\sum_{j=0}^{\ell-1}(x_{q,u+j} - s_{q,j})^2}$$

Ready-made operations provide an inductive bias; validation chooses what is useful.

Backup: eigen and topological representations

Eigen basis: lift the series into a trajectory matrix and retain low-rank structure.

$$H(x) = \begin{bmatrix} x_1 & x_2 & \cdots & x_{T-L+1} \\ x_2 & x_3 & \cdots & x_{T-L+2} \\ \vdots & \vdots & \ddots & \vdots \\ x_L & x_{L+1} & \cdots & x_T \end{bmatrix} \approx U_r \Sigma_r V_r^{\mathsf{T}}$$

Topology: embed delayed coordinates, then summarise persistent geometric structure.

$$\mathbf{z}_t = (x_t, x_{t+\tau}, \ldots, x_{t+(d-1)\tau}) \longrightarrow \{(b_j, d_j)\}_j \longrightarrow \phi(x)$$

Typical choices: embedding window / delay, retained rank, filtration and persistence features.



Backup: the kernel selector step by step

$$K_y[i,j] = 1\{y_i = y_j\}, \qquad C = I - \frac{1}{n}11^\intercal, \qquad \widetilde{K} = CKC$$

$$a_m = \max\left(0, \frac{\langle \widetilde{K}_m, \widetilde{K}_y \rangle_F}{\|\widetilde{K}_m\|_F \|\widetilde{K}_y\|_F}\right), \qquad c_m = \frac{\mathrm{tr}(K_m)}{\|K_m\|_F}$$

$$\mathbf{w}^{(t+1)} = \Pi_{\Delta_M}\left[\mathbf{w}^{(t)} + \eta\left(\mathbf{a} - \lambda\mathbf{c} - 2\mu R\mathbf{w}^{(t)}\right)\right]$$

$$K_{\text{test,train}} \in \mathbb{R}^{n_{\text{test}} \times n_{\text{train}}}, \qquad f(x) = \sum_{i=1}^{n_{\text{train}}} \alpha_i y_i \, k_w(x, x_i) + b$$

Then threshold small weights and renormalise. Dense storage grows quadratically in sample count.



Backup: kernel models and initial pipelines

A standalone kernel classifier

```
from fedot_ind.core.kernel_learning.estimators import (
    KernelEnsembleClassifier,
)
clf = KernelEnsembleClassifier(
    generator_names=("statistical_summary",
                     "wavelet_extractor"))
clf.fit(X_train, y_train)
probs = clf.predict_proba(X_test)
```

A separate initial-population path

```
from fedot_ind.core.kernel_learning import (
    integration as ki,
)

builder = ki.KernelInitialPopulationBuilder(
    task_type="classification")
specs = builder.build_specs(
    clf.kernel_importance_)
```

Generator importance → single-representation specs + a feature-union spec → candidate graphs

Inspect specs before building. Register Industrial operations before materialising graph nodes.



Backup: using Industrial on labelled sensor data

Prepared input: training windows, class labels, and separate test recordings.

```python
from fedot_ind.api.main import FedotIndustrial

model = FedotIndustrial(
    problem="classification", metric="f1",
    timeout=5, n_jobs=2)
model.fit((X_train, y_train))
labels = model.predict((X_test, None))
probs = model.predict_proba((X_test, None))
metrics = model.get_metrics(
    target=y_test, metric_names=["f1"])
model.shutdown()
```

Input / output contract

X_train: windows × channels × time
y_train: one label per window

labels: predicted classes
probs: class probabilities
metrics: evaluation against y_test

Before windowing:
split by machine or session.

Forecasting uses a time split.

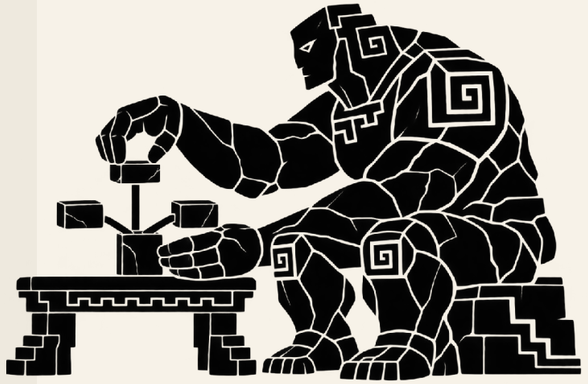


Backup: constructing an Industrial candidate

A hand-built candidate makes the search space concrete.

```python
from fedot.core.pipelines.pipeline_builder import PipelineBuilder
from fedot_ind.integration.fedot.extensions import (
    industrial_extension_scope,
)

with industrial_extension_scope():
    candidate = (PipelineBuilder()
        .add_node("wavelet_basis")
        .add_node("quantile_extractor_torch")
        .add_node("rf")
        .build())
```



Operation names and parameters → typed graph → fit and validate → candidate score

Search can replace a node, change a parameter, or explore a different branch structure.

Backup: forecasting hourly electricity load

Seven days of history → a 24-hour vector forecast. Fit on earlier windows only.

$$\mathbf{X}_t = (y_{t-L+1}, \dots, y_t), \qquad \mathbf{Y}_t = (y_{t+1}, \dots, y_{t+h}), \qquad \widehat{\mathbf{Y}}_t = f(\mathbf{X}_t)$$

```python
from fedot_ind.core.kernel_learning.estimators import (
    KernelEnsembleForecaster,
)
model = KernelEnsembleForecaster(
    forecast_horizon=24, head_type="kernel_ridge")
model.fit(X_train, Y_train)
Y_hat = model.predict(X_next)
```

Input windows: 168 steps
Targets: 24 future steps

Rolling time split
Naive / seasonal baseline

$$\mathrm{RMSE} = \frac{\|Y - \widehat{Y}\|_F}{\sqrt{Nh}}, \qquad Y, \widehat{Y} \in \mathbb{R}^{N \times h}$$



Backup: what the generated solution contains

LLM-written data glue surrounds fixed FEDOT training / prediction templates.

```
# Illustrative generated task-specific glue
train = pd.read_csv("train.csv")
target = train.pop("Response")
ids = test.pop("id")
train = train.drop(columns=["id"])

# Split and preprocess without test labels
model = train_model(X_fit, y_fit)
score = evaluate_model(model, X_val, y_val)
p = automl_predict(model, X_test)
```

Inspect the artifacts

solution.py
pipeline / model files
submission.csv
logs, metrics and report

Customer IDs preserve row identity.
Probability columns must match
the positive class.

A successful execution is necessary; target correctness and independent evaluation are separate checks.

Backup: documentation RAG versus model building

“How do I configure time-series forecasting?”

Researcher route:

retrieve documents
→ grade relevance
→ generate a grounded answer
→ assess usefulness / grounding
→ revise or return

“Train a 24-hour load forecaster on these files.”

AutoML route:

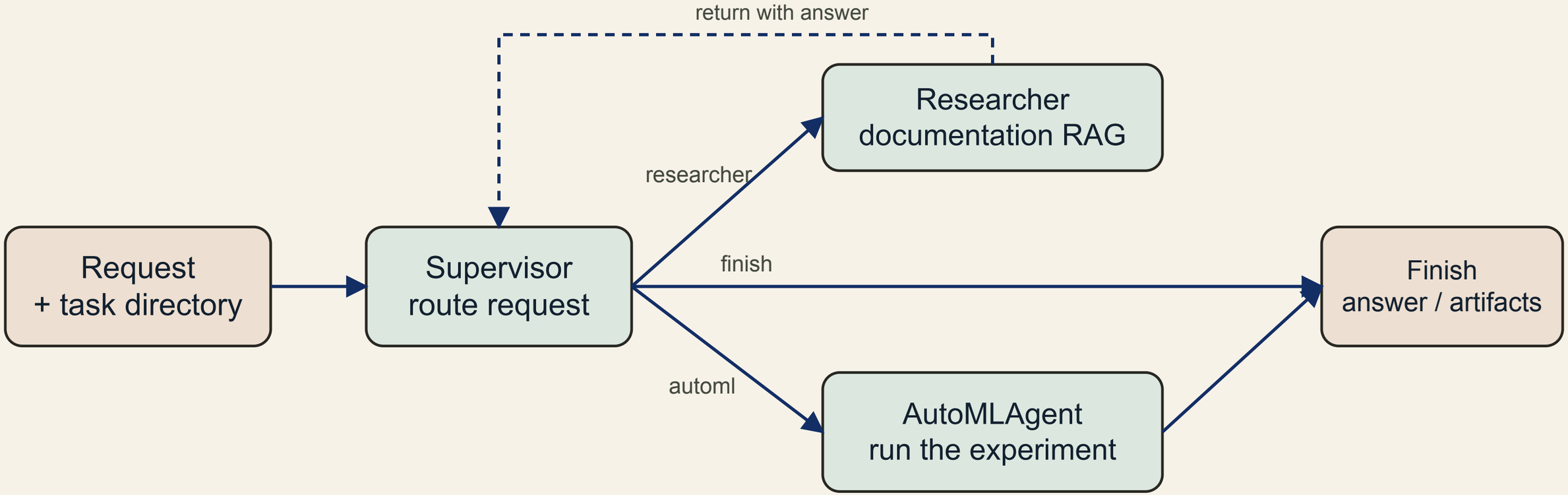
inspect task and data preview
→ generate configuration / code
→ execute FEDOT
→ inspect metrics / artifacts
→ report the run



Documentation explains an API.
A trained artifact solves the supplied task.

Backup: FEDOT.LLM agent routing

FedotAI composes a routing graph and a specialised AutoML subgraph.

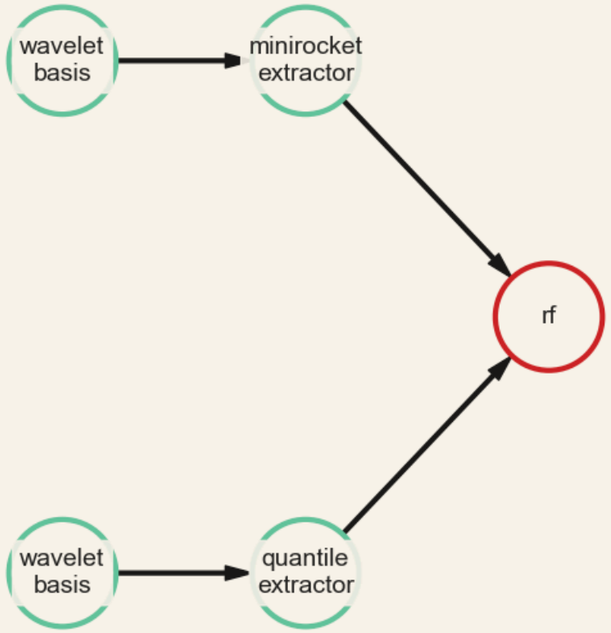


Automate task setup before FEDOT searches candidate models.

Shared messages connect the agents; the AutoML subgraph manages code and execution state.

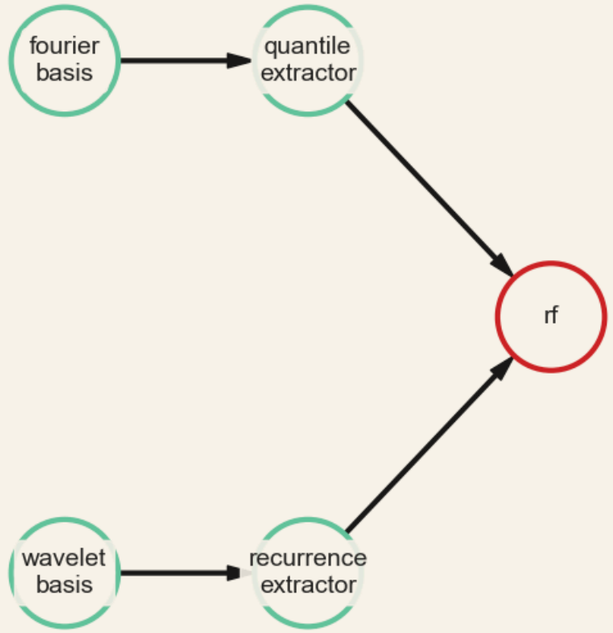
Backup: two more saved Industrial graphs

Herring · 5 operations



Wavelet → MiniROCKET
Wavelet → quantiles
Feature vectors meet at a random forest.

SmallKitchenAppliances · 5 operations



Fourier → quantiles
Wavelet → recurrence features
Different inductive biases, one classifier.

Backup: from inspection signal to anomaly intervals

Magnetic inspection: map model outputs back to intervals on the asset.

Predicted intervals

Expert-marked intervals



Position along the inspection route →

1. Represent the signal

Extract windows and signal features.

2. Group the patterns

Reported example: topology + HDBSCAN.

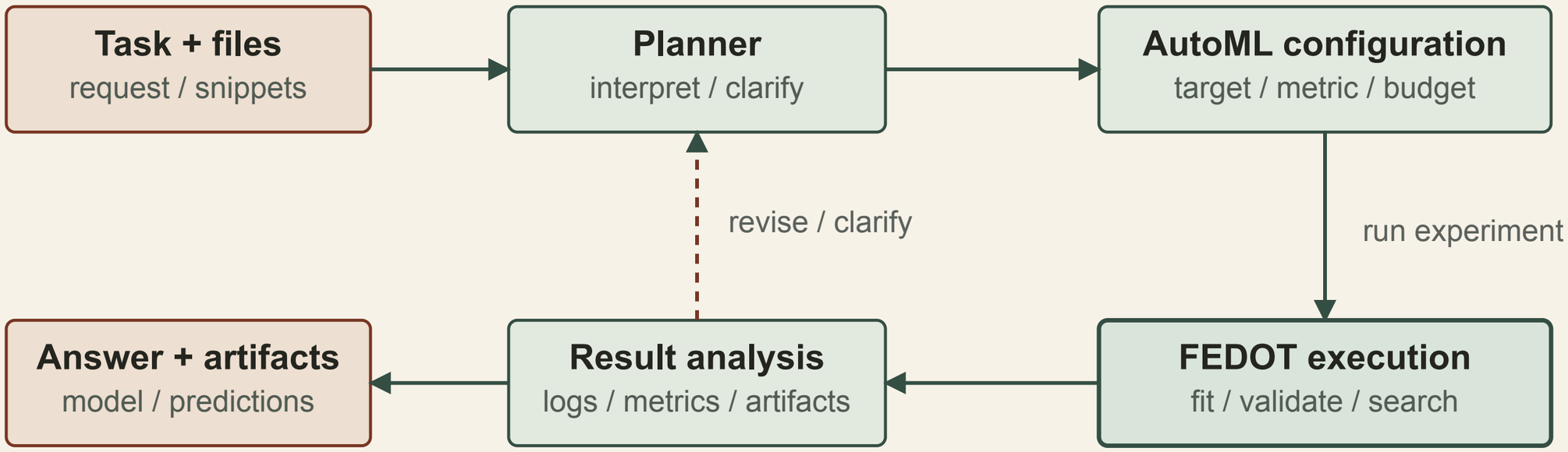
3. Recover intervals

Map window outputs back to the route.

Evaluate missed intervals, false alarms and localisation error.

Backup: FEDOT.LLM proposal: close the task loop

A proposed assistant connects the request to an experiment and its evidence.





The LLM manages the experiment. FEDOT evaluates model candidates.